Fundamentos de Programación

Tema 3: Instrucciones de Control en Python



Docente 2025/2026: Daniel Castro Pereiro
Apuntes originales: Efrén Arias Jordán y David Mera Pérez

Dpto Electrónica e Computación

Programación Estructurada

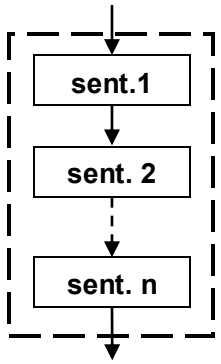
Programación estructurada

- Usa las técnicas de
  - **Estructuras de control** de flujo
  - **Diseño descendente**
  - **Recursos abstractos**: instrucciones y estructuras de datos de alto nivel
- Características
  - Claridad
  - Estilo estándar

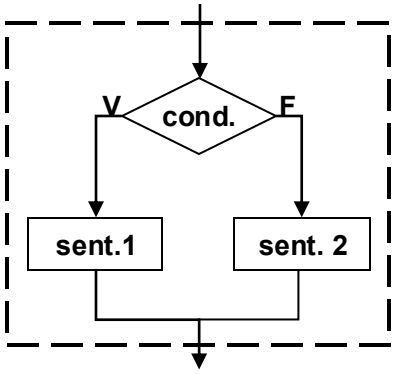
Programa Propio

- Definición
  - **Único punto de entrada** en el flujo de instrucciones
  - **Único punto de salida** en el flujo de instrucciones
  - Para cada instrucción del programa, existe un **camino** del flujo de instrucciones entre la entrada y la salida que pasa por ella.
  - Todas las instrucciones son **ejecutables**
  - No existen **lazos** (o bucles) **infinitos** (sin salida)

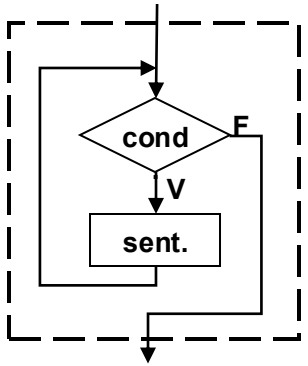
- Existen 3 estructuras de control fundamentales:
  - Secuencial (**`BLOCK`**)
  - Alternativa (**`IF-ELSE`**)
  - Repetitiva (**`WHILE`**)



`BLOCK`



`IF-ELSE`



`WHILE`

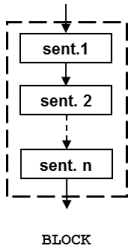
Estructura secuencial (BLOCK)

- Consiste en ejecutar una sentencia a continuación de otra.

Implementación en Python

- Se pueden agrupar varias sentencias para formar una única sentencia, denominada "**sentencia compuesta**", de la siguiente manera:

```
sent.1
sent.2
sent.3
...
sent.n
```



BLOCK

- En Python, lo que "**delimita**" el bloque es que todas tengan el mismo **nivel de indentación**, es decir, el mismo número de espacios por la izquierda.
- Aunque el número de espacios puede ser cualquiera que se desee, ha de ser el mismo para todas las sentencias que componen el bloque secuencial. Es costumbre que este número de espacios sea múltiplo de **4**.
- Hay que tener en cuenta que el "**bloque principal**" en Python ha de tener **cero espacios** de indentación, y que sólo los bloques que aparezcan dentro de otras estructuras de control irán indentados.
- En los ejemplos que hemos visto hasta este momento, todos los programas tienen un solo bloque, que es el "**principal**" y por tanto todos llevan cero espacios de indentación
- Tipicamente cada sentencia se escribe en una línea
- Se puede usar "**punto y coma**" para separar sentencias si están en la misma línea, pero en esta asignatura no usaremos esa característica y pondremos siempre cada sentencia en una línea separada.

Estructura Alternativa/Condicional

- Haciéndolo así, no es necesario poner un punto y coma para separarlas, y en particular **no es necesario poner punto y coma** al final de cada línea
- Los programas que hemos aprendido a construir hasta el momento presentan siempre una misma secuencia de acciones:
  1. Se piden datos al usuario: inicializamos variables (**asignando a variables con `input`**).
  2. Se efectúan cálculos (procesamos datos) con los datos introducidos por el usuario, guardando el resultado en variables (**mediante asignaciones**).
  3. Se muestran por pantalla los resultados almacenados en variables (resultados) (**mediante la función `print()`**).
- No obstante, es posible alterar el flujo de ejecución de los programas para hacer que:
  - tomen decisiones a partir de los datos y/o resultados intermedios y, se ejecuten **ciertas sentencias y otras no**
  - tomen decisiones a partir de los datos y/o resultados intermedios y, se ejecuten ciertas sentencias **más de una vez**.

Estructura Alternativa/Condicional

Un programa ilustrativo: resolución de ecuaciones de primer grado

- Diseñemos un programa para resolver cualquier ecuación de la recta $ax + b = 0$ donde **x** es la incógnita.
- Siguiendo el esquema de los programas que sabemos hacer, procederemos así:
  1. Pediremos el valor de **a** y el valor de **b** (que supondremos de **tipo flotante**).
  2. Calcularemos el valor de **x** como **–b/a**
  3. Mostraremos por pantalla el valor de **x**

```
#Tema3_01
print("Programa para la resolución de una ecuación de la recta ax+b=0")

#pedimos los valores
a=float(input('Dame el valor de a '))
b=float(input('Dame el valor de b '))

#calculamos la raíz de la ecuación
x=-b/a
print("el resultado es ", x)
```

¿Por qué este programa no es correcto en todos los casos?

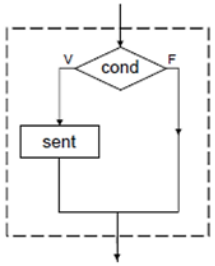
No existe solución si a=0

Estructura Alternativa/Condicional

- Permite elegir entre dos alternativas, según sea **True** o **False**, la condición que se evalúa. Existen dos subtipos

Alternativa simple

- Si la "`condición`" es **True** se ejecuta la "`sentencia`". Si no, no se ejecuta ninguna acción.



Implementación

```
if cond:
    sent
```

- Observad los siguientes detalles:
  - La condición será una expresión booleana cuyo resultado será `True` o `False`.
  - La condición finaliza con dos puntos
  - La sentencia o conjunto de sentencias a ejecutar dentro del bloque condicional deben de ir indentadas

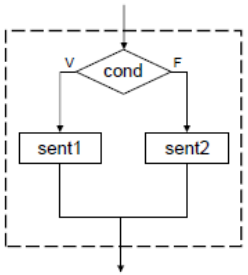
```
#El programa obtiene la nota por algún medio
if nota>=5.0:
    print("Prueba superada")
```

Alternativa doble

- Si la "`condición`" es **True** se ejecuta la "`sent1`". Si no, se ejecuta la "`sent2`".

Implementación

```
if cond:
    sent1
else:
    sent2
```



- Observad los siguientes detalles:
  - Las sentencias a ejecutar deben de ir indentadas
  - La palabra `else` ha de finalizarse con dos puntos y ha de tener el mismo nivel de indentación que la palabra `if`

```
#El programa inicializa num por algún medio
if num%2==0:
    print(" Es par ")
else:
    print(" Es impar ")
```

Estructura Alternativa/Condicional

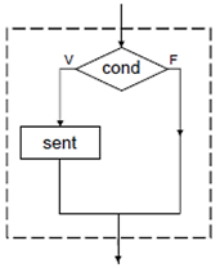
- ¿Cómo resolveríamos entonces el problema de la ecuación de la recta?

Alternativa simple

```
print("Programa para la resolución de una ecuación de primer grado ax+b=0")

#Tema3_02. Pedimos los valores
a=float(input('Dame el valor de a '))
b=float(input('Dame el valor de b '))

#calculamos la raíz de la ecuación
if a!=0:
    x=-b/a
    print("el resultado es ", x)
#No muestra nada en el caso de que a==0
```



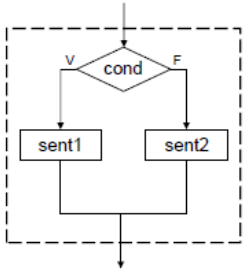
Alternativa doble

```
print("Programa para la resolución de una ecuación de primer grado ax+b=0")

#Pedimos los valores
a=float(input('Dame el valor de a '))
b=float(input('Dame el valor de b '))

#calculamos la raíz de la ecuación
if a!=0:
    x=-b/a
    print("el resultado es ", x)
else: #a==0
     print("No existe ecuación, ya que a vale 0")

#Muestra por pantalla el resultado o que no existe ecuación, si a==0
```



Estructura alternativa /Condicional

Más ejemplos

Programa que lee dos números y dice si el primero es múltiplo del segundo.

```
#Tema3_05
n1 = int(input("Introduce un valor entero: "))
n2 = int(input("Introduce otro valor entero: "))
if n1%n2==0:
    print(n1,"es múltiplo de ",n2)
else:
    print(n1,"No es múltiplo de ",n2)
```

Programa que lee un número y dice si es par o impar.

```
num = int(input("Introduce un número entero: "))
if num%2==0:
    print(" Es par ")
else:
    print(" Es impar ")
```

Programa que lea tres números enteros (a, b y c). Si a es menor que b, entonces incrementar c en una unidad. Finalmente, visualizar todos los valores.

```
#Tema3_06
a = int(input("Introduce un valor entero para a: "))
b = int(input("Introduce un valor entero para b: "))
c = int(input("Introduce un valor entero para c: "))
if a<b:
  c=c+1
print("\nLos valores finales son\na->",a,"\nb->",b, "\nc->",c)
```

Programa que lea tres números enteros (a, b y c). Si a es menor que b, entonces incrementar c en una unidad, sino si a es mayor o igual que b entonces decrementar c en una unidad. Finalmente, visualizar todos los valores.

```
#Tema3_06
a = int(input("Introduce un valor entero para a: "))
b = int(input("Introduce un valor entero para b: "))
c = int(input("Introduce un valor entero para c: "))
if a<b:
  c=c+1
else:
  c=c-1
print("\nLos valores finales son\na->",a,"\nb->",b, "\nc->",c)
```

Estructura alternativa /Condicional

Operador ternario

- El operador ternario en programación es una forma de hacer operaciones condicionales con 3 parámetros en una sola línea...

- Para que tengáis un ejemplo, el condicional **if** básico sería algo así:

```
nota=6
if nota>=5:
    resultado=”Aprobado";
else:
    resultado=”Suspenso";
```

- Utilizando el operador ternario, seria así:

```
nota=6
resultado="Aprobado" if nota>5 else "Suspenso"
```

- Esta estructura nos permite ahorrar código, lo que significa escribir menos líneas y hacerlas más sencillas.

- La sintaxis sería:

```
resultado = valor_si if condicion else valor_no
```

- Veamos un ejemplo que nos imprimirá el número introducido sólo si es mayor que 50, si no, imprimirá 0

```
c = int(input("Introduce un número: "))
r = c if (c>50) else 0
print(r)
```

Con el ejemplo de par/impar...

```
num=int(input("Introduce un número entero: "))
resultado = 'par' if num%2==0 else 'impar'
print(resultado)

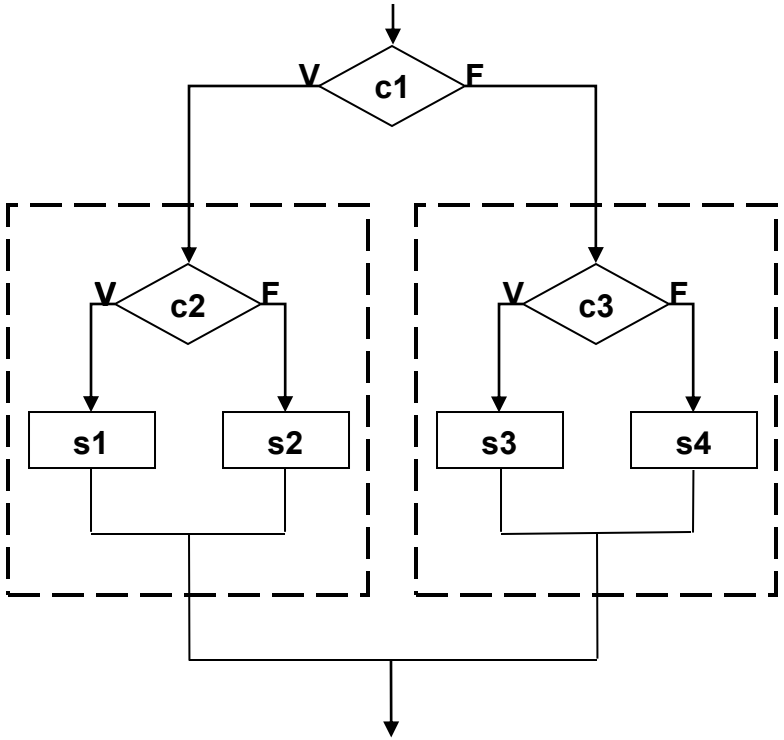
otra forma:
 print('par') if num%2==0 else print('impar')
```

- E incluso lo podemos utilizar para asignar varios valores a varias variables al mismo tiempo.

```
#Tema3_08_1
import random
a,b,c,d = (1,2,3,4) if (random.randint(0,100)>50) else (4,
3,2,1)
print (a,b,c,d)
```

Anidamiento de estructuras alternativas

- Las estructuras alternativas pueden "**anidarse**", esto es, donde debería ir `sent1` o `sent2` en la alternativa doble, podemos poner otra estructura alternativa. Tanto en la alternativa simple (if) como en la doble (if-else)
- El anidamiento puede complicarse más, si en lugar de **`s1`**, **`s2`**, **`s3`** o **`s4`** en dicha figura, incluimos otra estructura alternativa. El lenguaje no pone límite al nivel de anidamiento, que puede ser tan profundo como se quiera.
- En la práctica, sin embargo, utilizar más de dos niveles resulta difícil de leer, y suele ser un síntoma de que se podía haber diseñado el algoritmo de otra forma, o de que parte de él puede ser extraído a una función



Implementación

```
if c1:
   if c2:
       s1
   else:
       s2
else:
   if c3:
       s3
   else:
       s4
```

Anidamiento de estructuras alternativas

- Python proporciona una **estructura multialternativa** (**`if-elif-else`**), que permite elegir entre varias alternativas según el resultado de diferentes expresiones booleanas.
- En realidad, puede implementarse mediante una serie de **`if-else`** anidados "**en cascada**", pero la sintaxis con **`if-elif-else`** resulta más legible, al reducir el anidamiento.

```
if cond1:
   sent1
elif cond2:
   sent2
. . .
elif condN:
   sentN
else:
   sentencia  #cualquier otro caso
```

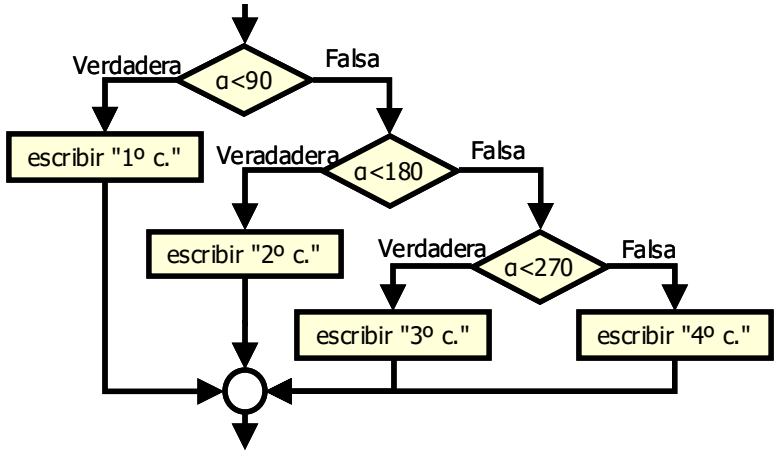
  - La idea general se leería como "si se cumple **cond1** haz **sent1**, si no, si se cumple **`cond2`** haz **`sent2`**, si no, si se cumple **`cond3`** haz **`sent3`**, etc.. y si no se cumple ninguna, haz **`sentencia`**",

Observaciones sobre la implementación:

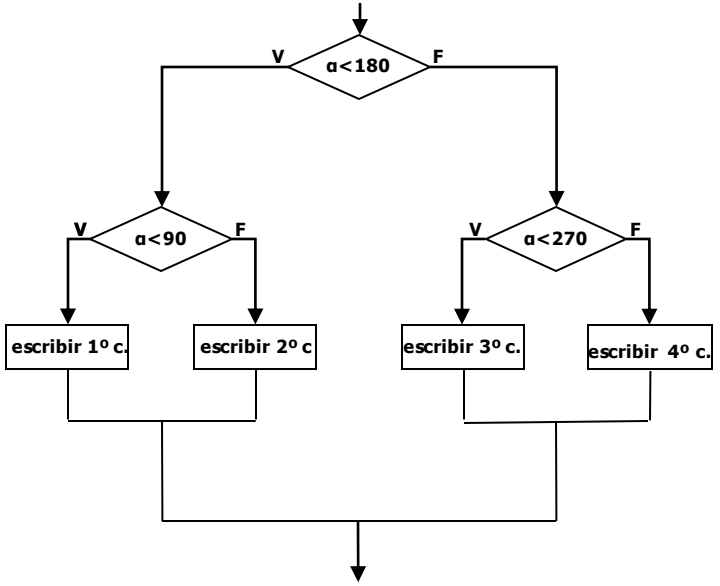
- La palabra **`elif`** es una contracción de **`else if.`**
- Cada uno de los **`elif`** ha de ir forzosamente alineado con el **`if`** inicial, así como el **`else`** final.
- Se pueden poner tantos **`elif`** como se necesiten.
- El **`else`** final es opcional. Si no se pone, y ninguna de las condiciones se ha cumplido, entonces no se ejecutará ninguna sentencia de las incluidas en el **`if-elif.`**
- Como siempre, cualquiera de las **`sent`** del código anterior puede ser un bloque de sentencias.
- Solo se ejecutará, como mucho, un camino.

Anidamiento de estructuras alternativas

- **Ejemplo**: Determinar en qué cuadrante está un ángulo alfa, positivo, menor de 360º.



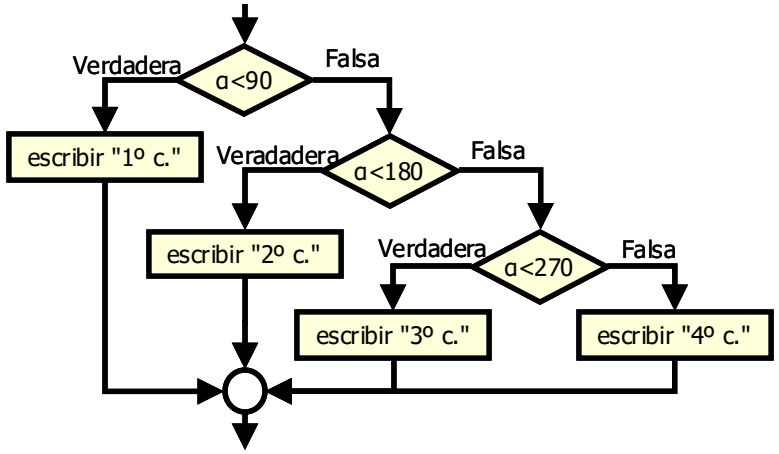

```
alfa=float(input("introduzca un ángulo entre 0 y 360 "))
if alfa < 90:
    print("1º cuadrante")
elif alfa < 180:
    print("2º cuadrante")
elif alfa < 270:
    print("3º cuadrante")
else:
    print("4º cuadrante")
```

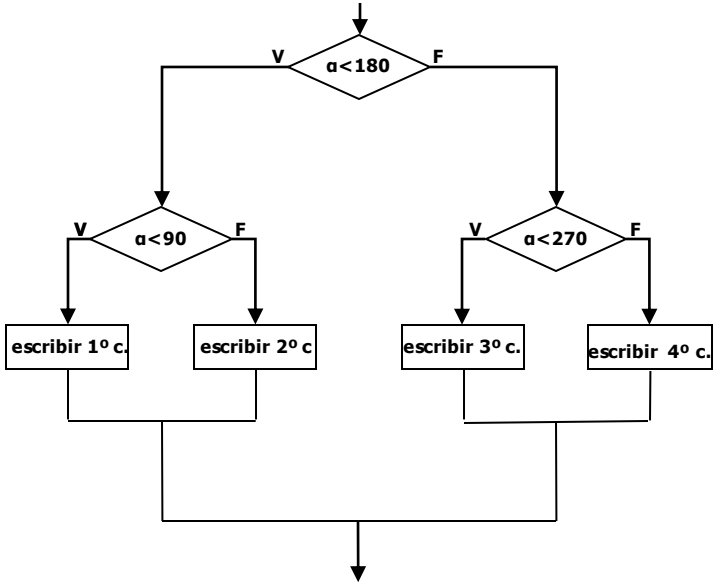



```
alfa=float(input("introduzca un ángulo entre 0 y 360 "))
if alfa < 180:
    if alfa<90:
        print("1º cuadrante")
    else:
        print("2º cuadrante")
else:
    if alfa < 270:
        print("3º cuadrante")
    else:
        print("4º cuadrante")
```

Anidamiento de estructuras alternativas

- **Ejemplo**: Determinar en qué cuadrante está un ángulo alfa, positivo, menor de 360º.






```
#Tema3_18
print("Programa para la calcular en que cuadrante esta un de
terminado grado")

from math import floor
#pedimos los valores
grados=float(input('Dame el ángulo (en grados) '))

cuadrante = floor(grados)%360//90 # resultado 0,1,2 o 3
if cuadrante<2:
    if cuadrante==0:
        print('El ángulo ', grados, ' está en el primer cuad
rante')
    else:
        print('El ángulo ', grados, ' está en el segundo cua
drante')
else:
    if cuadrante==2: print('El ángulo ', grados, ' está en e
l tercer cuadrante')
    else: print('El ángulo ', grados, ' está en el cuarto cu
adrante')
```

Anidamiento de estructuras alternativas

- El siguiente ejemplo pide al usuario una nota numérica y escribe la correspondiente nota cualitativa.
- Otro uso típico de la estructura multialternativa es en el procesamiento correspondiente a la elección realizada por el usuario cuando se le presentan a éste un conjunto de posibles opciones.

```
#Tema3_19
nota = float(input("Introduzca la nota del examen: "))
if nota >= 0 and nota < 5:
    print("Suspenso")
elif nota >= 5 and nota < 7:
    print("Aprobado")
elif (nota >= 7 and nota < 9):
    print("Notable")
elif (nota >= 9) and (nota <= 10):
    print("Sobresaliente")
else:
    print("Nota no válida")
```

- Otra Forma:

```
#Tema3_20
nota = float(input("Introduzca la nota del examen: "))
if (0<=nota< 5):
    print("Suspenso")
elif (5<=nota< 7):
    print("Aprobado")
elif (7<=nota< 9):
    print("Notable")
elif (9<=nota <= 10):
    print("Sobresaliente")
else:
    print("Nota no válida")
```

- Dados dos números enteros, en las variables **a** y **b**, determinar su relación de orden.

```
#Tema3_21
a=int(input("Introducir un entero a:"))
b=int(input("Introducir otro entero b:"))
    #Se averigua la relación de orden, usando sentencias if-
else
if a > b:
    print("a > b, ya que ",a," > ",b)
elif a < b:
    print("a < b, ya que ",a," < ",b)
else:
    print("a = b, ya que ",a," = ",b)
```

Anidamiento de estructuras alternativas

Otro ejemplo: resolución de ecuaciones de segundo grado

- En esta ocasión vamos a resolver ecuaciones de segundo grado, que son de la forma $ax^2 + bx + c = 0$ donde **x** es la incógnita.

```
#Tema3_07
print("Programa para la resolución de una ecuación de seg
undo grado ax2+bx+c=0")
from math import sqrt

#pedimos los valores
a=float(input('Dame el valor de a '))
b=float(input('Dame el valor de b '))
c=float(input('Dame el valor de c '))

#calculamos las raíces de la ecuación de segundo grado
x1=(-b+sqrt(b**2-4*a*c))/(2*a)
x2=(-b-sqrt(b**2-4*a*c))/(2*a)
```

¿Por qué este programa no es correcto en todos los casos?

- No existe solución real si el radicando es negativo,
- Si a=0, sería una ecuación de primer grado
- Si a=0 y b=0, no existe ecuación

Posible solución!

```
#Tema3_08
print("Programa para la resolución de una ecuación de seg
undo grado ax2+bx+c=0")
from math import sqrt
#pedimos los valores
a=float(input('Dame el valor de a '))
b=float(input('Dame el valor de b '))
c=float(input('Dame el valor de c '))

#calculamos las raíces de la ecuación de segundo grado
if a!=0:
    radicando = b**2-4*a*c
    if radicando>=0:
        x1=(-b+sqrt(b**2-4*a*c))/(2*a)
        x2=(-b-sqrt(b**2-4*a*c))/(2*a)
        print("Las soluciones son X1=%.3f y X2=%.3f" %(x1
,x2))
    else:#radicando negativo
        print("Las soluciones no son reales, porque el ra
dicando es negativo")
else:#a==0
    if b!=0:
        print('La solucion es unica y es %.3f'%(-c/b))
    else: #a==0 y b==0
        print(' No existe ecuacion, puesto que a==0, b==0
 y por tanto c==0')
```

Anidamiento de estructuras alternativas

Menús de usuario

- Permite seleccionar uno de los casos de los que disponga el programa.

Ejemplo: Hacer un programa que a partir del radio de una circunferencia calcula su diámetro, perímetro o área.

```
#Tema3_22
#programa que a partir del radio de una circunferencia
calcula su diámetro, perímetro o área.
from math import pi
radio=float(input('Dame el radio de un círculo: '))

#Menu
print('\tEscoge una opción: ')
print('\t\ta)Calcular el diámetro. ')
print('\t\tb)Calcular el perímetro. ')
print('\t\tc)Calcular el área. ')

opcion=input('\tTeclea a, b o c y pulsa return: ')
#Todos los menús de usuario los realizaremos de esta fo
rma: mostramos las opciones posibles y leemos la respue
sta del usuario
if opcion=='a':
    diametro=2*radio
    print('El diámetro es %8.3f' %diametro)
elif opcion=='b':
    perimetro=2*pi*radio
    print('El perímetro es %5.2f' %(perimetro))
elif opcion=='c':
    area=pi*radio**2
    print('El área es %5.2f' %(area))
else:
    print('Error en la opción. Tienes que teclear
 a, b o c')
    print('Tu has tecleado %c'%opcion)
```

Problema de ejemplo:

- Si en las variables **a** y **b** tenemos registrados dos números decimales, y en la variable **op** se encuentra uno de los cuatro operadores aritméticos (**+, –, x, /**), realizar la operación aritmética (**a op b**)

Anidamiento de estructuras alternativas

```
#Tema3_23
#Se piden los operandos a y b, y el operador
a=float(input("Introducir el primer operando a: "))
op=input("Introducir el operador (+,-,x,/) ")
b=float(input("Introducir el segundo operando. "))

#Se realiza la operación aritmética, usando solamente sen
tencias if-else
if op=='+':
      print("El resultado de hacer a + b es: {0}+{1}={2}"
.format(a,b,a+b))
elif op=='-':
      print("El resultado de hacer a - b es: {0}-{1}={2}"
.format(a,b,a-b))
elif op == 'x':
      print("El resultado de hacer a x b es: {0}x{1}={2}"
.format(a,b,a*b))
elif op == '/':
    if b!=0:
      print("El resultado de hacer a / b es: {0}/{1}={2}"
.format(a,b,a/b))
    else:#b==0
      print(" no se puede dividir por 0 ")
else:
    print("Error en el operador")
```

La sentencia match

- Una sentencia **`match`** toma una expresión y compara su valor con patrones sucesivos dados como uno o más bloques de casos.
- Solo se ejecuta el primer patrón que coincide y también puede extraer componentes (elementos de secuencia o atributos de objeto) del valor en variables.
- La estructura **`Match-Case`** sólo es válida a partir de la versión de Python 3.10, y tiene la siguiente sintaxis:

```
match subject:
    case <pattern_1>:
        <action_1>
    case <pattern_2>:
        <action_2>
    case <pattern_3>:
        <action_3>
    case _:
        <action_wildcard>
```

Anidamiento de estructuras alternativas

- En la que se evalúa si el contenido de la variable **`subject`** es igual al del patrón (**`<pattern_1>`**) situado después de la primera instrucción **`case`**.
  - Si es cierto, se ejecutará el código debajo de este y el flujo de ejecución saldrá de la estructura.
  - En el caso de que sea falso, se evaluará el patrón después del siguiente **`case`** hasta que se encuentre uno que sea válido o se evalúen todos los casos de forma negativa.

```
color = '#FF0000'

match color:
    case '#FF0000':
        print('🔴')
    case '#00FF00':
        print('🟢')
    case '#0000FF':
        print('🔵')
```

- ¿Pero qué ocurre si el valor que comparamos no existe entre las opciones disponibles?
- Si lo queremos controlar, se debe emplear el comodín _, siendo importante que este se incluya en último lugar, ya que en caso contrario no se evaluarían los **`cases`** debajo de este.

```
color = '#AF549B'

match color:
    case '#FF0000':
        print('🔴')
    case '#00FF00':
        print('🟢')
    case '#0000FF':
        print('🔵')
    case _:
        print('Unknown color!')
```

Anidamiento de estructuras alternativas

```
#Tema3_23_1
#Se piden los operandos a y b, y el operador
a=float(input("Introducir el primer operando a: "))
op=input("Introducir el operador (+,-,x,/) ")
b=float(input("Introducir el segundo operando. "))
result=None
match op:
    case '+':
        result = a + b
    case '-':
        result = a - b
    case '*':
        result = a * b
    case '/':
        if b!=0:
            result = a / b
    case _:
        print('Error en el operador')

if result is not None:
    print('El resultado de la operación es 'f'{a}{op}{b}
= {result}')
elif b==0:
    print(" no se puede dividir por 0")
```

Evaluación con cortocircuitos

Ejemplos conceptuales

- **Expresiones unidas por AND**: Evalúo de izquierda a derecha. Si alguna expresión es falsa, todo es falso por lo que no necesito evaluarlo. Ejemplo: para realizar una receta necesito todos los ingredientes. Si me falta el primero, ya no puedo hacer la receta.


AND

AND


- **Expresiones unidas por OR**: Evalúo de izquierda a derecha. Si alguna expresión es verdadera, toda la condición pasar a ser verdadera por lo que no necesito evaluar las otras partes. Ejemplo. Si pincho una rueda, la cambio yo o llamo a una grúa. No hago las 2 cosas.


OR


Evaluación con cortocircuitos

- Al calcular el resultado de una expresión lógica, Python evalúa lo justo hasta conocer el resultado: **cortocircuitos.** La evaluación de expresiones lógicas tiene algo especial.

```
if a==0 or 1/a > 1:
```

¿Puede provocar una división por cero? **No, nunca.** Observa que si **a==0**, el primer término del `or` es `True` por lo que la evaluación del `or` siempre será `True` con cualquier otro valor en el segundo término por lo que Python no lo evalúa y se ahorra así un esfuerzo innecesario.

Ejemplo:

```
a = 0
b = 5
if a == 0 or 1/a > 1:
  print("No se produce una división por cero.")
else:
  print("Esta línea nunca se ejecuta.")
```

```
if a!=0 and 1/a > 1:
```

Si **a** es cero, el valor de **a!=0** es `False`, así que ya no se procede a evaluar la segunda parte de la expresión, ya que un `and` requiere que todos los términos sean `True` para ser globalmente `True`

Ejemplo:

```
a = 0
b = 5
if a != 0 and 1/a > 1:
  print("Esta línea nunca se ejecuta.")
else:
  print("No se produce una división por cero.")
```

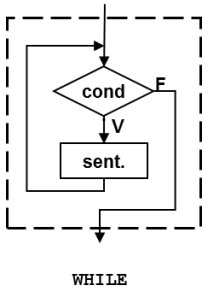
Estructura repetitiva while

Estructura repetitiva while

- Una estructura repetitiva es un bloque de sentencias (denominado **cuerpo del bucle**) que se va a ejecutar varias veces. Se incluye una **condición**, que regula si se seguirá repitiendo o no.
- Las instrucciones dentro del bloque modificarán los valores de algunas de las variables que forman parte de la condición, haciendo que en algún momento la evaluación de esa condición cambie a **`False`**.

Implementación

```
while cond:
    sent
```



WHILE

Funcionamiento

- La **`condición`** es lo primero que se evalúa
  - Si el resultado es **`False`**, se abandona el bucle sin ejecutar **`sent`**.
  - Si es **`True`**, se ejecuta **`sent`** y tras ello se vuelve otra vez a la condición, para evaluarla de nuevo.

```
#Tema3_24
n = int(input("Numero entero: "))
a = n
digitos = 0
while a!= 0:
    # sent
    digitos = digitos + 1
    a = a//10   # Se modifica la variable a que se
va reduciendo.

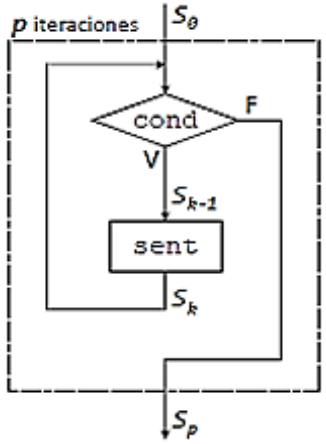
# En algún ciclo el valor de a será 0 y la
# condición a!= 0 será falsa
print(n, "tiene", digitos, "dígitos")
```

Estado	n	dígitos	a	a!=0
S_0	62574	0	62574	True
S_1	62574	1	6257	True
S_2	62574	2	625	True
S_3	62574	3	62	True
S_4	62574	4	6	True
S_5	62574	5	0	False

Estructura repetitiva while

- Un programa (o algoritmo) es una secuencia de instrucciones tal que, seguidas paso a paso, proporciona una **secuencia ordenada de estados finita** en la que el último estado (o estado final) contiene la solución del problema a resolver para cada entrada válida. Un bucle nos permite avanzar esa secuencia de estados

```
# Estado inicial, S0
while cond: # iteración k (k = 1, 2, ..., p)
    # Estado Sk-1
    sent
    # Estado Sk
# Estado final, Sp
```



Reglas de construcción del while

1. Inicializar todas las variables para obtener el estado inicial.
2. Determinar la **condición de parada** del bucle `while`.
   - Para establecer la condición se tendrá en cuenta que el estado final tiene que contener la salida que se espera del bucle. *La evaluación de la condición como True, implica la ejecución del bucle y como False su parada.*
3. Crear las sentencias (bloque del while) que nos permitan avanzar en nuestro objetivo (secuencia de estados) iteración tras iteración. Estas sentencias deben de evolucionar el estado (modificando las variables) de forma que, en algún momento, el bucle termine (cond==False).
   - *Si la condición SIEMPRE es True, entonces se genera un bucle infinito*
   - *Si la condición NUNCA es True entonces nunca se ejecuta el bucle*

- En general**, no modificamos las variables que contienen los datos de entrada al bucle**. Si es necesario para avanzar hacia la solución, previamente se asignarán estos valores a otras variables y serán éstas últimas las que se modifiquen.

Estructura repetitiva while

- **Ejemplo:** Escribir un programa que permita obtener en pantalla la secuencia de enteros 1, 3, 6, 10, 15, ... que no superen un valor umbral positivo dado

```
#Tema3_25. Establecer el estado inicial
umbral = int(input("Umbral: "))
termino = 1
cantidad = 2
while termino <= umbral:
    print (termino) # escribe en pantalla el término
 actual
    termino = termino + cantidad
    cantidad = cantidad + 1
print()
```

Estado	umbral	termino	cantidad
S_0	25	1	2
S_1	25	1+2=3	3
S_2	25	3+3=6	4
S_3	25	6+4=10	5
S_4	25	10+5=15	6
S_5	25	15+6=21	7
S_6	25	21+7=28	8

Un problema de ejemplo: cálculo de sumatorios

- Empezaremos por un programa que calcula la suma de los **N** primeros números, es decir, un programa que calcula el **sumatorio**

```
#Tema3_26. Programa que hace el sumatorio de los N pri
meros números naturales
sumatorio=0
i=1
N=int(input("hasta que número deseas hacer el sumatori
o? "))
while i<=N:
    sumatorio=sumatorio+i #o los que es lo mismo sumat
orio+=i . Esto acumula la suma de los números
naturales desde 1 hasta N.
    i+=1 #pasar al siguiente número natural en la
siguiente iteración del bucle.
print('El sumatorio de los %d primeros numeros es %d'%
(N,sumatorio))
```

- *Diseña un programa que calcule el sumatorio entre n y m, donde n y m son enteros que deberá introducir el usuario por teclado*

Estructura repetitiva while

Otro programa de ejemplo: requisitos en la entrada

- Escribir un programa que pida un número **`float`** y calcule su raíz cuadrada

```
#Tema3_28. Importamos sqrt de math
from math import sqrt

#pedimos un valor positivo de x
x= float(input("Introduce un numero positivo: "))

#que pasa si el usuario introduce un número negativo?
print ("La raíz de %5.2f es %5.2f"%(x,sqrt(x)))
```

```
Introduce un numero positivo: -125
Traceback (most recent call last):
  File "c:/Users/Usuario/AATRABAJO/DOCENCIA/Curso
20_21/PYTHON/CODIGOS/Tema 3/Tema3_20.py", line 15, in
<module>
    print ("La raíz de %5.2f es %5.2f"%(x,sqrt(x)))
ValueError: math domain error
```

- Una forma de solucionarlo es hacer una **lectura validada**, que consiste en inicializar la variable a leer de manera que la condición de un **`while`** en donde encerramos la lectura sea **`True`**

```
#Tema3_29. Importamos sqrt de math
from math import sqrt
#inicializamos la variable a leer

x=-1
while x<0:
#pedimos un valor positivo de x
    x= float(input("Introduce un numero positivo: "))
#que pasa si el usuario introduce un número negativo?
#que no sale del bucle while hasta que le dé un numero p
ositivo de x

print ("La raíz de %5.2f es %5.2f"%(x,sqrt(x)))
```

- *Diseña un programa que solicite la lectura de un número entre 0 y 10 (ambos inclusive).*

Estructura repetitiva while

Bucle infinito…

```
Papá, que es un bucle infinito?
No sé, pregúntale a tu madre
Mamá, que es un bucle infinito?
No sé, pregúntale a tu padre
Papá, que es un bucle infinito?
No sé, pregúntale a tu madre
Mamá, que es un bucle infinito?
No sé, pregúntale a tu padre
```

…

- Los bucles son muy útiles a la hora de confeccionar programas, pero también son peligrosos si no andas con cuidado: es posible que no finalicen nunca. Estudia este programa y verás qué queremos decir:

```
i=0 # Tema3_32
while i<10:
    print(i)
```

- La condición del bucle siempre se satisface: dentro del bucle nunca se modifica el valor de **i**, y si **i** no se modifica, jamás llegará a valer 10 o más.

- El ordenador empieza a mostrar el número 0 una y otra vez, sin finalizar nunca. Es lo que denominamos un **bucle sin fin** o **bucle infinito**.

```
# Tema3_33. Programa que hace el sumatorio de los N prim
eros números naturales
sumatorio=0
i=1
N=int(input("hasta que número deseas hacer el sumatorio?
 "))
while i<=N:
    sumatorio=sumatorio+i #o los que es lo mismo sumator
io+=i
    i*=1
print('El sumatorio de los %d primeros numeros es %d'%(N
,sumatorio))
```

```
hasta que número deseas hacer el sumatorio? 100
Traceback (most recent call last):
  File "C:/…/CODIGOS/bucle infinito2.py", line 7, in
<module>
    i*=1
KeyboardInterrupt
```

Estructura repetitiva while

```
#Tema3_34. Programa que calcula el diámetro, perímetro
o área de una circunferencia.

from math import pi
radio=float(input('Dame el radio de un círculo: '))
#Menu con lectura validada: Para validar la entrada del
usuario y asegurarse de que elija una opción válida.
opcion=' '
while opcion<'a' or opcion>'c':
    print('\tEscoge una opción: ')
    print('\t\ta)Calcular diámetro. ')
    print('\t\tb)Calcular perímetro. ')
    print('\t\tc)Calcular área. ')

    opcion=input('\tTeclea a, b o c y pulsa return: ')

if opcion=='a':
    diametro=2*radio
    print('El diámetro es {0}.'.format(diametro))
elif opcion=='b':
    perimetro=2*pi*radio
    print('El perímetro es %5.2f' %(perimetro))
elif opcion=='c':
    area=pi*radio**2
    print('El área es %5.2f' %(area))
```

while-else

- Python permite una clausula **`else`** con el **`while`**

```
cont=0
while cont<=10:
    print(cont)
    cont+=1
else:
    print('Hemos llegado a 10')
```

- Las sentencias contenidas en la clausula **`else`** se ejecutan solo una vez cuando la condición del **`while`** pasa a ser **`False`**
- La clausula **`else`** del while NO es común en otros lenguajes de programación
- En general existen pocos escenarios en los que sea necesario usarla (una parte asociados a la rotura de bucles con **`break`**) y en esta asignatura no la fomentaremos

Estructura repetitiva for

Estructura repetitiva `for-in`

- Es muy frecuente que aparezca la necesidad de realizar un bucle para el que se conoce de antemano el número de iteraciones a realizar.
- En Python, además del bucle **`while`**, también existe el bucle **`for`** y puede ser usado para implementar esta idea de "**bucle controlado por contador**".

```
for <variable> in <serie_de_valores>:
   acción
```

- El **`for`** de Python es mucho más versátil que en otros lenguajes, y puede usarse también para obtener los caracteres sucesivos de una cadena o para que vaya tomando una secuencia de valores arbitraria, a través de las listas

```
#Tema3_25.
#Recorriendo una lista
for nombre in ['Pepe', 'Ana', 'Juan']:
   print('Hola, %s' %nombre)

#Recorriendo caracteres de una cadena

for letra in 'abcdefghi':
   print(letra,end=" ")
```

- **`for-in`** se emplea habitualmente en bucles en los que se conoce de antemano el número de iteraciones
- Python proporciona **`range()`** que genera una secuencia (lista) de enteros entre un valor inicial y uno final, y con un paso (step)
- **`range()`** nos permite iterar un número concreto de veces sobre un bloque de sentencias con facilidad

Estructura repetitiva for

- La sintaxis general de **range** es **range(inicial, final, paso)** siendo:
  - **inicial** el valor del primer entero de la secuencia. Puede omitirse, y entonces recibirá automáticamente el valor cero.
  - **final** el valor del primer entero "fuera" de la secuencia. Este valor ya **no formará parte** de la secuencia de enteros generada, que se detendrá en el entero anterior a éste.
  - **paso** es la distancia entre los enteros que se van generando. Puede omitirse y entonces recibirá el valor uno.

```
range(1,10,2) # genera 1, 3, 5, 7, 9
range(1,10,-1) #no genera ninguna secuencia
range(10,1,-1) #genera 10, 9, 8, 7, 6, 5, 4, 3, 2
>>> list(range(2,10))
[2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(0,3))
[0, 1, 2]
>>> list(range(-3,3))
[-3, -2, -1, 0, 1, 2]
>>> list(range(-10,-1))
[-10, -9, -8, -7, -6, -5, -4, -3, -2]
>>> list(range(3,-1,-1))
[3, 2, 1, 0]
list(range(10,1,-3))
[10, 7, 4]
```

- *¿Qué parámetros habría que pasarle a **range** para que genere todos los múltiplos positivos de 3 y menores de 100?*

Estructura repetitiva for

- La sintaxis del **`for-in range`** es la siguiente:

```
for i in range(inicial,final,paso):
    sentencia1
    sentencia2
```

- La variable **`i`** del **`for`**, es la variable de **control del bucle**. El estado de esta variable está determinado por **`range`** y su cambio de estado por el **`paso`** de éste

  - **Ejemplo1 :** Diseña un programa que muestre la raíz n-ésima de un número leído por teclado, con **n** tomando valores entre 2 y 100.

```
# Tema3_36
numero=float(input('Dame un numero: '))

for n in range(2,101):
    #print('La raiz {0}-ésima de {1} es {2}'.format(n,numero,numero**(1/n)))
    print('La raiz %d-ésima de %5.2f es %e' %(n,numero,numero**(1/n)))
```

Estructura repetitiva for

- **Antes, while:** Escribir un programa que permita obtener en pantalla la secuencia de enteros 1, 3, 6, 10, 15, ... que no superen un valor umbral positivo dado

```
#Tema3_25. Establecer el estado inicial
umbral = int(input("Umbral: "))
termino = 1
cantidad = 2
while termino <= umbral:
    print (termino) # escribe en pantalla el término
 actual
    termino = termino + cantidad
    cantidad = cantidad + 1
print()
```

- **Ahora: for.** Dado un entero **n** (>=0), escribir un programa que permita obtener en pantalla los **n** primeros términos de la secuencia de enteros 1, 3, 6, 10, 15, ...

```
# Tema3_37b
n = int(input("Entero (>=0): "))
termino = 1
for cantidad in range(2, n+2, 1):
  print(termino, end=" ") # escribe en pantalla el    término actual
  termino = termino + cantidad
print()
```

```
Otra forma:
n = int(input("Entero (>=0): "))
termino = 1
for cantidad in range(1, n+1, 1):
    print(termino, end=" ") # escribe en p
antalla el término actual
    termino = termino + (cantidad + 1)
print()
```

Estructura repetitiva for

for-in como forma compacta de ciertos while

- Ciertos bucles se ejecutan un número de veces fijo y conocido a priori.

```
#Tema3_38. Programa que hace el sumatorio de los 1000
 primeros números naturales
sumatorio=0
i=1
while i<=1000:
    sumatorio+=i
    i+=1
print('El sumatorio de los %d primeros numeros es %d'
%(1000,sumatorio))
```

- En este patrón la variable **i** suele denominarse **índice** del bucle.

Podemos expresar de forma compacta este tipo de bucles con un **for-in** siguiendo este otro patrón:

```
for i in range(valor_inicial, valor_final,+1):
```

- El programa de cálculo del sumatorio de los 1000 primeros números se puede expresar ahora de este modo:

```
#Tema3_39.
sumatorio=0
for i in range(1,1001,+1):
    sumatorio+=i
print('El sumatorio de los %d primeros numeros es %d'%(1
000,sumatorio))
```

Estructura repetitiva for

Números primos

- Un número primo es aquel número mayor que 1 que sólo es divisible por 1 y por sí mismo. Dado un numero **n**, si todos los restos de dividir n entre 2,3,…,n-1 son distintos de 0, entonces **n** es primo

```
# Tema3_40.
n=-1 #validamos el valor de n>1
while n<2:
    n=int(input('Dame un numero entero positivo y >1:
'))

for divisor in range(2,n):
    print('{0} entre {1} '.format(n,divisor),end=' ')
     print('es %d con resto %d'%(n//divisor,n%divisor)
)
```

```
 Dame un numero entero positivo y >1: 7
 7 entre 2  es 3 con resto 1
 7 entre 3  es 2 con resto 1
 7 entre 4  es 1 con resto 3
 7 entre 5  es 1 con resto 2
 7 entre 6  es 1 con resto 1
```

- ¿cómo nos aseguramos de que todos los restos son distintos de cero? Una posibilidad es contarlos y comprobar que hay exactamente **n-2** restos no nulos.
- Otra posibilidad es hacer que deje de buscar divisores, en cuanto encuentre uno que ya lo divida. Para ello usaremos un **`while`** en vez de un **`for-in`**

```
# Tema3_41.
n=-1 #n=int(input('Dame un numero entero positivo y >1 '))
while n<2:
n=int(input('Dame un numero entero positivo y >1 '))

divisor=2
es_primo=(n%divisor!=0) #Esto devuelve True or False
#print('Pruebo con divisor ',divisor)
divisor=3

#no hace falta probar con los divisores entre 2 y n-1. Basta probar hasta n//2
while divisor<=n//2 and es_primo:
  es_primo=(n%divisor!=0)
  divisor+=2

if es_primo: #Si es primo es True; es primo; si es False: No
  print(f'El numero {n} si es primo')
else:
  print(f'El numero {n} NO es primo')
```

Estructura iterativa

```
#La mejor versión para comprobar si un número es primo

n=-1
while n<2:
    n=int(input('Dame un numero entero positivo y >1 '))
divisor=2
es_primo=(n%divisor!=0) #Devuelve True or False

while divisor < n**0.5 and es_primo:
    print('Pruebo con divisor ',divisor)
    es_primo=(n%divisor!=0)
    divisor+=2


if es_primo:
    print('El numero %d SI es un numero primo '%n)
else:
    print('El numero %d NO es un numero primo '%n)
```

Rotura de bucles: `break`

- Hay una sentencia que permite abortar la ejecución de un bucle desde cualquier punto del mismo: **`break.`** Su uso está totalmente desaconsejado, por ir en contra del principio de programación estructurada

Reevaluación de bucles: `continue`

- La sentencia **`continue`** devuelve el control al inicio del bucle dejando sin ejecutar lo que tenga por debajo

La razón por la que se verifica hasta la raíz cuadrada del número es porque, **si un número no es primo y tiene un divisor mayor que su raíz cuadrada, entonces necesariamente también tiene un divisor menor que su raíz cuadrada**. Por lo tanto, no es necesario verificar más allá de la raíz cuadrada para determinar si un número es primo o no.

Estructura iterativa

- Rotura de bucles (**break**). Ejemplo conceptual
  - Tengo que hacer ejercicios y mientras no los termine todos no puedo ir a jugar
    - Si me llama mi mejor amig@ dejo de hacer los ejercicios y me voy (termino el bucle aunque no haya terminado)

```
while tengo_ejercicios:
```



```
    if me_llama_amig@:
        break
```





Estructura iterativa

- Reevaluación de bucles (**continue**). Ejemplo conceptual
  - Tengo que hacer ejercicios y mientras no los termine todos no puedo ir a jugar
    - Si me llama mi amig@ y me dice que los ejercicios son menos de los pensados, paro (dejando lo que estuviese haciendo) y vuelvo a evaluar qué hacer.

```
while tengo_ejercicios:
```



```
    if me_llama_amig@ con actualización de ejercicios:
        continue
```



Estructura iterativa

Anidamiento de estructuras iterativas

- *Diseña un programa que pida un número y nos muestre por pantalla los números primos entre 2 y el introducido.*

```
# Tema3_42.
print('programa pida un número y nos muestre por pantalla l
os números primos entre 2 y el que hemos introducido')
tope=-1
while tope<2:
    tope=int(input('Dame un numero entero positivo y >2 '))

print(2,end=' ')
for n in range(2, tope+1):
    divisor=2
    es_primo=(n%divisor!=0)
    divisor=3
    while divisor <= n**0.5 and es_primo:
        es_primo=(n%divisor!=0)
        divisor+=2
    if es_primo:
        print(n,end=' ')
```

- Tiene bucles anidados (un **`while`** dentro de un **`for-in`**), pero está claro qué hace cada uno de ellos

Escribe un programa que lea un número n por teclado y después lea n números $x_1, x_2, \ldots, x_n$ y calcule su media y producto:

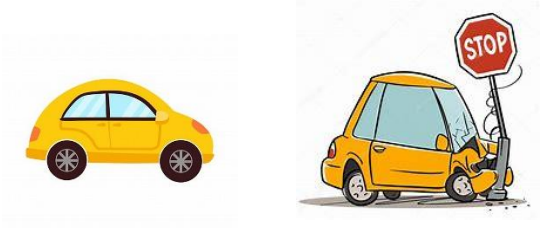
```
# Tema3_43. Usando un while
n = int(input ("Introduce número de elementos: "))
i = 0
media = 0.0
producto = 1.0
while i < n:
   xi = float(input("Introduce elemento: "))
   media = media + xi
   producto = producto * xi
   i = i + 1
media = media / n
print ("Media=", media)
print ("Producto=", producto)

# Tema3_44. Usando un for
n = int(input ("Introduce número de elementos: "))
media = 0.0
producto = 1.0
for i in range(0,n,1):
   xi = float(input("Introduce elemento: "))
   media = media + xi
   producto = producto * xi
media = media / n
print ("Media=", media)
print ("Producto=", producto)
```

Tratamiento de excepciones

- Ejemplo conceptual I
  - En un trayecto en coche (programa) pueden surgir accidentes (errores) y tengo que estar "atento" (**try/except**) para solucionarlos. Si tengo un accidente puedo tratarlo llamando al seguro, ambulancia, policía, grúa, etc.

try:



except:



Tratamiento de excepciones

- Ejemplo conceptual II
  - Si llego bien a mi casa (sin accidentes) puedo avisar a mi familia de que todo fue bien (**else**)

try:



except:







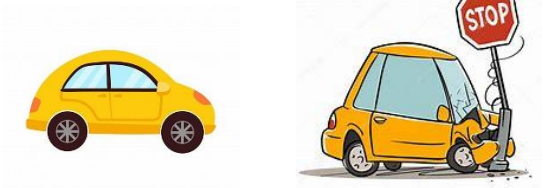
else:



Tratamiento de excepciones

- Ejemplo conceptual III
  - Pase lo que pase (**finally**) dejaré el coche apagado y cerrado

try:



except:



else:



finally:



Tratamiento de excepciones

- Ejemplo conceptual IV
  - ¿Es conveniente hacer las mismas acciones para todos los accidentes (pinchazo, rotura cristal, choque, animal, etc.)? Es preferible actuar según el tipo de accidente (**diferentes bloques de except** dedicados a cada posible accidente).

try:



except AccidentePinchazo:



except AccidenteAnimal:



except AccidenteConHeridos:





Tratamiento de excepciones

- Ejemplo conceptual V
  - Incluso puedo tener un plan para un accidente que no tenga contemplado que solo usaré si mi accidente no encaja con los otros planes (**except:**)

try:



except AccidentePinchazo:



except AccidenteAnimal:



except AccidenteConHeridos:





except :







Tratamiento de excepciones (try-except)

Captura y tratamiento de excepciones

- Aunque la sentencia sea sintácticamente correcta, puede generar un error cuando se ejecuta.
- Estos errores se conocen como **excepciones**, algunas son: divisiones por cero, cálculos matemáticos inadecuados (raíces de valores negativos), operar con tipos incompatibles, etc.
- Para evitar las excepciones se utiliza **`try-except`**, que puede entenderse como ***«ejecuta estas sentencias y, si se comete un error, ejecuta estas otras».***

- La forma de uso del **`try-except`** es :

```
try:
     acciones pot. erróneas 1..n
except:
     acción para tratar el error 1
     …
     acción para tratar el error n
else:
    acción realizada correctamente
[finally: acción final]
```

```
#Tema3_09.Ejemplo ecuación recta
print("Programa para la resolución de una ecuación de prim
er grado ax+b=0")
#pedimos los valores
a=float(input('Dame el valor de a '))
b=float(input('Dame el valor de b ’))
try:
    x=-b/a
    print("el resultado es ", x)
except: #hay un un error, a==0
    if b!=0:
        print("La ecuación no tiene solución")
    else:
        print("La ecuación  tiene infinitas soluciones.
Cualquier valor de x satisface la ecuacion”)
```

Tratamiento de excepciones

Semántica:

- 1.- Se ejecuta el bloque **`try`**
  - Si no ocurre ninguna excepción dentro de él:
    - i. Los bloques **`except`** se ignoran.
    - ii. Se ejecuta el bloque **`else`**, en caso de que este exista.
  - Si ocurre una excepción durante la ejecución del bloque **`try`**:
    - Se ignoran el resto de las instrucciones del bloque **`try`** y la ejecución salta al bloque **`except`** que referencia al error que ha ocurrido
    - Si no existe dicho bloque **`except`**, el programa termina con un mensaje de error.
- 2.- Se ejecuta el bloque **`finally`** tanto si se capturó alguna excepción como si no. Incluso si en algún lugar previo se ejecutó un **`return`**.
    - La sentencia **return** se mostrará en el tema de las funciones

- Ejemplo:

```
#Tema3_10
import math
unNumero = int(input("Por favor deme un entero "))

try:
    print(math.sqrt(unNumero))
except:
    print("Valor incorrecto para la raíz cuadrada")
    print("Se usa el valor absoluto en sustitución")
    print(math.sqrt(abs(unNumero)))
```

- Otro ejemplo:

```
#Tema3_11
import math
import sys
try:
    resultado= math.factorial(2.4)
except:
    print("Ha ocurrido algo imprevisto.", sys.exc_info
()[0])
else:
    print("El factorial es", resultado)
```

Tratamiento de excepciones

- Las excepciones tienen un «tipo» asociado y podemos distinguir el tipo de excepción para actuar de diferente forma en función del tipo de error detectado.

```
>>> 1/0
Traceback (most recent call last):
  File "<pyshell#0>", line 1 1/0
ZeroDivisionError: division by zero
```

```
>>> from math import sqrt
>>> sqrt (-1)
Traceback (most recent call last):
  File "<pyshell#2>", line 1 sqrt (-1)
ValueError: math domain error
```

- Algunas excepciones comunes
  - **`NameError`**: no puede encontrar un nombre local o global (variable, función, etc.). Por ejemplo no importamos math
  - **`TypeError`**: a una función se le pasa un argumento de tipo inapropiado.
  - **`ValueError`**: el tipo es correcto pero un valor inapropiado.
  - **`ZeroDivisionError`**: División por cero.
  - **`FileNotFoundError`**: cuando el archivo o diccionario no existe.

Tratamiento de excepciones

- Una declaración **try** puede tener más de un **except**, pero sólo se ejecutará una de ellas. Un mismo bloque **except** puede atrapar varios tipos de excepciones, lo cual se hace especificando los nombres de la excepciones separados por comas a continuación de la palabra **except**.

```
#Tema3_12
try:
    x=30
    y=int("a")
    #y=0
    #y=65

#cambiad los valores de y
    result= x / y
except ZeroDivisionError:
    print("¡División por cero!")
except NameError:
    print("¡Variable no existe!")
except ValueError:
    print("¡Error en asignación!")
except:
    print("Error")
else:
    print(result)
#finally:
#   print("ejecutando la cláusula finally")
```

- También es posible que un programador cause una excepción en tiempo de ejecución utilizando la instrucción **raise**.
- Por ejemplo, en lugar de llamar a la función raíz cuadrada con un número negativo, podríamos haber comprobado primero el valor y luego haber generado nuestra propia excepción.
- El siguiente fragmento de código muestra el resultado de la creación de una nueva excepción **RuntimeError**.
- El programa se interrumpiría de todos modos, pero ahora la excepción que causó la interrupción es algo creado explícitamente por el programador

```
#Tema3_13
import math
unNumero = int(input("Por favor deme un entero "))
if unNumero < 0:
    raise RuntimeError("Usted no puede usar un número
negativo")
else:
    print(math.sqrt(unNumero))
```

Tratamiento de excepciones

Comprobaciones por tipo

- En esta clase de comprobaciones nos interesa el tipo del dato que vamos a tratar de validar, Python nos indica el tipo de una variable usando la función **`type(variable).`**
- Por ejemplo, para comprobar que una variable contenga un tipo entero podemos hacer:

```
#Tema3_14
i="2"
if type(i) != int:
    raise TypeError ("i debe ser del tipo int")
```

- Es posible comprobar el tipo de nuestra variable contra una secuencia de tipos posibles.

```
i="2"
if type(i) not in (int, float, complex):
    raise TypeError ("i debe ser numérico")
```

```
# Tema3_15. Resolución de ecuación de segundo grado con excepcio
nes
print("Programa para la resolución de una ecuación de segundo gr
ado ")
from math import sqrt
#Modificar la petición comprobando tipos
a=float(input('Dame el valor de a '))
b=float(input('Dame el valor de b '))
c=float(input('Dame el valor de c '))
#calculamos las raíces de la ecuación de segundo grado
try:
    x1=(-b+sqrt(b**2-4*a*c))/(2*a)
    x2=(-b-sqrt(b**2-4*a*c))/(2*a)
    if x1==x2:
        print('Solución: x={0:.3f}'.format(x1))
    else:
        print("Las soluciones son X1=%.3f y X2=%.3f" %(x1,x2))

except ZeroDivisionError:#aqui entra si a==0
    if b!=0:
        print('La solución es unica y es %.3f'%(-c/b))
    else:
        print('La ecuación tiene infinitas soluciones')

except ValueError:
    print("Las soluciones no son reales, porque el radicando es
negativo")
```

Tratamiento de excepciones

Otro ejemplo: requisitos en la entrada

- Para evitar que en el acto de lectura leamos un tipo no válido, usaremos una excepción, con dos posibles valores, **`ValueError`** y **`EOFError`**, como se muestra en el siguiente ejemplo:

```
#Tema3_30
edad = -1 # Inicializamos con un valor invalido
while edad <= 0:
    try:
        edad = int(input('Introduce tu edad en años: '))
        if edad <= 0:
            print( 'Tu edad ha de ser positiva' )
    except (ValueError, EOFError):
        print( 'Respuesta invalida’ )
```

```
Introduce tu edad en años: -23
Tu edad ha de ser positiva
Introduce tu edad en años: hh
Respuesta invalida
Introduce tu edad en años: 25
```

- También podríamos hacerlo dentro de un bucle del que no salimos mientras el valor leído no sea del tipo correcto

```
#Tema3_31. Invocamos a una función, exactamente igual q
ue con las funciones predefinidas, pero esta está escri
ta por nosotros

def lee_entero():
   while True:
       entrada = input("Escribe un numero entero: ")
       try:
           entrada = int(entrada)
           return entrada
       except ValueError:
           print ("La entrada es incorrecta: escribe un
 numero entero")

#Programa principal que llama a la función definida por
 nosotros
entero=lee_entero()
print(entero)
```

Una estrategia de diseño

Una estrategia de diseño: refinamientos sucesivos

- Es lógico que cuando estés aprendiendo a programar te cueste gran esfuerzo construir mentalmente un programa tan complicado, pero posiblemente sea porque sigues una aproximación equivocada: **no debes intentar construir mentalmente todo el programa de una vez.**
  - **Primero haz una versión sobre papel** que resuelva el problema de forma directa y trasládalo al ordenador.
  - **Analiza tu programa**…¿realmente resuelve el problema planteado? ¿existen errores en tiempo de ejecución?, ¿hay datos especiales para los que el cálculo debe ser diferente?
  - Cada vez que te plantees una de estas preguntas y tengas una respuesta, modifica el programa en consecuencia. **No hagas más de un cambio cada vez.**
  - **Si el programa ya funciona correctamente** para todas las entradas posibles y eres capaz de anticiparte a los posibles errores de ejecución, ¡enhorabuena!, ya casi has terminado.
- Nadie es capaz de hacer un programa suficientemente largo de una sentada, desde la primera línea a la última…
- Un **error frecuente** es tratar de diseñar el programa directamente sobre el ordenador, escribiéndolo a bote pronto.

Una estrategia de diseño

- Es más, hay estudiantes que se atreven a empezar con la escritura de un programa sin haber entendido bien el enunciado del problema que se pretende resolver…

El síndrome «A mí nunca se me hubiera ocurrido esto»

- **Programar** es una actividad que **requiere** un **gran esfuerzo intelectual**, pero sobre todo es radicalmente diferente de cualquier otra que conoces.

- Debes tener en cuenta:

  - La solución final muchas veces esconde la línea de razonamiento que permitió llegar a ese programa concreto (**refinamientos sucesivos**)

- La solución que se te presenta sigue la línea de razonamiento de una persona concreta: **el docente**

- **Tenéis que encontrar vuestra propia línea de razonamiento**